



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF CSE

PROJECT ABSTRACT SUBMISSION

COURSE: SOA PROGRAMMING AND MICROSERVICES– (24SDCS03)

A.Y. 2026 - 2027

TITLE OF THE PROJECT	Library Fine Management System	
S. No.	University ID	Name
1	2420030573	MVS Aasrith
2	2420030014	V Sanjay Bhaargav
3	2420030012	M Rishikesh
Name of the Guide		Dr. Navyatha Ravi

Abstract:

Libraries manage large numbers of books and users, making it important to maintain accurate records of borrowed books, return dates, and fines. Manual fine calculation and record maintenance can be time-consuming and may lead to errors in tracking overdue books, outstanding amounts, and payment status. The Library Fine Management System is proposed as a simple and efficient solution to automate the management of library fines. The system enables authorized users to maintain borrower and book information, record borrowing and return transactions, calculate fines for overdue books, and monitor the payment status of fines.

To address these requirements, the proposed system follows a Service-Oriented Architecture (SOA) using a microservices-based approach. The application is divided into independent services so that major library operations can be developed, maintained, and tested separately. The system provides REST APIs for communication between services and uses a database for storing service-specific information. This approach improves modularity, scalability, maintainability, and ease of integration.

The first approach introduces a new technique, "Oppositional Aquila Optimizer-based Feature Selection with Deep Belief Network for Epileptic Seizure Detection" (OAOFS-DBNECD).

The Authentication Service manages user registration, login, and secure access using Spring Security and JSON Web Tokens (JWT). The Library Service manages book and borrower information along with borrowing and return records. The Fine Management Service calculates applicable fines based on overdue periods, maintains fine records, and updates payment status. A Notification Service can be used to provide reminders for overdue books and pending fines. These services communicate through REST APIs and are managed using service discovery and an API gateway.

The proposed system uses Java and Spring Boot for microservice development, MySQL for data storage, Spring Data JPA for database interaction, Netflix Eureka for service registration and discovery, and Spring Cloud Gateway for centralized API routing. Maven is used for project build and dependency management, while Postman is used for REST API testing. Swagger/OpenAPI can be used for API documentation, and JUnit with Mockito can be used for unit testing. GitHub and GitHub Actions support version control and continuous integration, while Docker can be used for containerizing the services.

The Library Fine Management System provides a structured and automated way to manage overdue books and associated fines. By separating the application into independent services, the system reduces complexity and allows individual services to be updated without affecting the complete application. The proposed solution improves accuracy in fine calculation, provides secure access to library operations, and offers a reliable platform for managing fine-related activities.

Collectively, the proposed microservices-based system provides a simple, secure, and maintainable solution for library fine management. It demonstrates the practical use of SOA principles, RESTful services, authentication, service discovery, API gateway routing, database management, testing, and continuous integration. The system can help library staff maintain transparent fine records and provide users with clear information about overdue books and outstanding payments.

Project Architecture/Project Overview:

The project consists of Authentication, Library, Fine Management, and Notification services. The services expose REST APIs and are connected through Spring Cloud Gateway, with Netflix Eureka providing service discovery. Each service maintains its own data using MySQL and communicates with other services through REST-based service calls. JWT-based authentication and role-based authorization secure the APIs. The system can be tested using Postman and JUnit, with GitHub Actions supporting automated build and testing.

